📘 Tuples in Python

Tuple Operations | All Built-in Tuple Methods/Functions

Introduction to Tuples

Definition: A tuple is a sequence of elements enclosed in parentheses (`( )`), where elements can be of different data types.

```
tup1 = (10, 20, 30, 40)
tup2 = ("apple", "banana", "cherry")
tup3 = (1, "hello", 3.14, True)
tup4 = ()               # empty tuple
tup5 = (5,)             # single element tuple (note the comma)
```

Important Points:

- Tuples are immutable (cannot be changed after creation)
- Elements in a tuple can be accessed using index positions
- Indexing starts from 0 (forward) and -1 (backward)
- Tuples can contain elements of different data types
- Tuples maintain the order of elements (ordered collection)
- A single element tuple must have a trailing comma, e.g., `(5,)`

Tuple Operations

1. Indexing

Purpose: Accesses an individual element of a tuple using its position

Syntax: `tuple[index]`

Parameters: index (compulsory) — position of the element

Return: A single element at the given index

- Forward indexing starts from 0 (left to right)
- Backward indexing starts from -1 (right to left)

- Index must be an integer, otherwise `TypeError` occurs
- Raises `IndexError` if the index is out of range

```
tup1 = (10, 20, 30, 40, 50)
print(tup1[0])    # 10 (first element)
print(tup1[4])    # 50 (last element)
print(tup1[-1])   # 50 (last element, backward index)
print(tup1[-5])   # 10 (first element, backward index)
# print(tup1[10])  # IndexError: tuple index out of range
```

2. Concatenation (+)

Purpose: Joins two or more tuples together

Syntax: `tuple1 + tuple2`

Parameters: Two or more tuples

Return: A new tuple combining all tuples

- Both operands must be tuples
- Creates a new tuple (doesn't modify original)

```
tup1 = (1, 2, 3)
tup2 = (4, 5, 6)
result = tup1 + tup2  # (1, 2, 3, 4, 5, 6)
```

3. Repetition / Replication (*)

Purpose: Repeats a tuple a specified number of times

Syntax: `tuple * n` or `n * tuple`

Parameters: tuple (compulsory), integer n (compulsory) — number of repetitions

Return: A new tuple repeated n times

- n must be an integer
- If n = 0, returns empty tuple

```
tup1 = (1, 2)
result = tup1 * 3  # (1, 2, 1, 2, 1, 2)
```

4. Membership Operators (in / not in)

Purpose: Checks if an element exists in a tuple

Syntax: `element in tuple` or `element not in tuple`

Parameters: element (compulsory) — item to search for

Return: Boolean (True/False)

- Case-sensitive for string elements
- `in` returns True if element is present
- `not in` returns True if element is not present

```
tup1 = (10, 20, 30, 40)
print(20 in tup1)       # True
print(50 in tup1)       # False
print(50 not in tup1)   # True
```

5. Slicing

Purpose: Extracts a portion of a tuple

Syntax: `tuple[start:end:step]`

Parameters:

- start (optional) — starting index (default: 0)
- end (optional) — ending index (excluded, default: length)
- step (optional) — step value (default: 1)

Return: A new sub-tuple

- All parameters are optional
- Negative indices can be used
- start < end for forward slicing

```
tup1 = (10, 20, 30, 40, 50, 60)
print(tup1[0:3])      # (10, 20, 30)
print(tup1[3:])       # (40, 50, 60)
print(tup1[::-1])     # (60, 50, 40, 30, 20, 10) (reverse)
print(tup1[::2])      # (10, 30, 50)
```

Traversing a Tuple Using Loops

Using for loop:

```
tup1 = (10, 20, 30, 40)
for item in tup1:
    print(item)
# Output: 10 20 30 40 (each on new line)
```

Using for loop with index:

```
tup1 = (10, 20, 30, 40)
for i in range(len(tup1)):
    print(tup1[i])
```

Using while loop:

```
tup1 = (10, 20, 30, 40)
i = 0
while i < len(tup1):
    print(tup1[i])
    i += 1
```

Built-in Functions / Methods

1. len()

Purpose: Returns the number of elements in a tuple

Syntax: `len(tuple)`

Parameters: tuple (compulsory)

Return: Integer (number of elements)

- Counts all elements in the tuple

```
tup1 = (10, 20, 30, 40, 50)
print(len(tup1))  # 5
```

2. tuple()

Purpose: Creates a tuple from an iterable (string, list, range, etc.)

Syntax: `tuple(iterable)`

Parameters: iterable (optional) — any iterable object

Return: A new tuple

- If no argument given, returns an empty tuple
- Converts string characters into tuple elements

```
print(tuple())              # ()
print(tuple("Hello"))       # ('H', 'e', 'l', 'l', 'o')
print(tuple([1, 2, 3]))     # (1, 2, 3)
print(tuple(range(5)))      # (0, 1, 2, 3, 4)
```

3. count()

Purpose: Counts occurrences of an element in a tuple

Syntax: `tuple.count(element)`

Parameters: element (compulsory) — item to count

Return: Integer (number of occurrences)

- Case-sensitive for string elements
- Returns 0 if element not found

```
tup1 = (10, 20, 10, 30, 10, 40)
print(tup1.count(10))  # 3
print(tup1.count(50))  # 0
```

4. index()

Purpose: Finds the first occurrence of an element

Syntax: `tuple.index(element, start, end)`

Parameters:

- element (compulsory) — item to search for
- start (optional) — starting index (default: 0)
- end (optional) — ending index (default: length)

Return: Integer (index of first occurrence)

- Raises ValueError if element not found
- Case-sensitive for string elements

```
tup1 = (10, 20, 30, 20, 40)
print(tup1.index(20))      # 1
print(tup1.index(20, 2))   # 3
# print(tup1.index(50))    # ValueError
```

5. sorted()

Purpose: Returns a new sorted list from the elements of the tuple

Syntax: `sorted(tuple, reverse=True/False)`

Parameters:

- tuple (compulsory) — the tuple to be sorted
- reverse (optional) — if True, sorts in descending order (default: False)

Return: A new sorted list (not a tuple)

- Does not modify the original tuple
- Returns a list, not a tuple
- To get a tuple, use `tuple(sorted(tup))`

```
tup1 = (40, 10, 30, 20)
new_list = sorted(tup1)
print(new_list)  # [10, 20, 30, 40]
print(tup1)      # (40, 10, 30, 20) (unchanged)

new_tuple = tuple(sorted(tup1))
print(new_tuple) # (10, 20, 30, 40)
```

6. min()

Purpose: Returns the smallest element in the tuple

Syntax: `min(tuple)`

Parameters: tuple (compulsory)

Return: The smallest element

- Works on numbers and strings (lexicographic)
- Raises ValueError if tuple is empty

```
tup1 = (40, 10, 30, 20)
print(min(tup1))  # 10
```

7. max()

Purpose: Returns the largest element in the tuple

Syntax: `max(tuple)`

Parameters: tuple (compulsory)

Return: The largest element

- Works on numbers and strings (lexicographic)
- Raises ValueError if tuple is empty

```
tup1 = (40, 10, 30, 20)
print(max(tup1))  # 40
```

8. sum()

Purpose: Returns the sum of all numeric elements in the tuple

Syntax: `sum(tuple)`

Parameters: tuple (compulsory) — must contain only numbers

Return: The sum of all elements

- Raises TypeError if tuple contains non-numeric elements
- Returns 0 for an empty tuple

```
tup1 = (10, 20, 30, 40)
print(sum(tup1))  # 100
```

Tuple Assignment

Definition: Tuple assignment (also called unpacking) allows multiple variables to be assigned values from a tuple in a single statement.

1. Basic Tuple Assignment

Purpose: Assign multiple variables at once from a tuple

Syntax: `var1, var2, ... = tuple`

Parameters: Number of variables must match number of elements in tuple

Return: None (assigns values to variables)

```
tup1 = (10, 20, 30)
a, b, c = tup1
print(a, b, c)  # 10 20 30
```

2. Swapping Two Variables

Purpose: Swap values of two variables without using a temporary variable

Syntax: `a, b = b, a`

```
a = 10
b = 20
a, b = b, a
print(a, b)  # 20 10
```

3. Using * to Collect Multiple Values

Purpose: Collect multiple values into a list

Syntax: `a, *b, c = tuple`

```
tup1 = (1, 2, 3, 4, 5)
a, *b, c = tup1
print(a)  # 1
print(b)  # [2, 3, 4]
print(c)  # 5
```

- The number of variables must match the number of elements (unless using *)

- Raises ValueError if the number of variables does not match
- Useful for returning multiple values from a function

Nested Tuples

Definition: A nested tuple is a tuple that contains other tuples as its elements.

```
nested_tup = ((1, 2, 3), (4, 5, 6), (7, 8, 9))
```

Accessing elements in nested tuples:

```
nested_tup = ((1, 2, 3), (4, 5, 6), (7, 8, 9))
print(nested_tup[0])      # (1, 2, 3)
print(nested_tup[1][2])   # 6
print(nested_tup[2][0])   # 7
```

Traversing a nested tuple:

```
nested_tup = ((1, 2, 3), (4, 5, 6), (7, 8, 9))
for inner in nested_tup:
    for element in inner:
        print(element, end=" ")
    print()
# Output:
# 1 2 3
# 4 5 6
# 7 8 9
```

Tuple assignment with nested tuples:

```
nested_tup = ((1, 2), (3, 4))
(a, b), (c, d) = nested_tup
print(a, b, c, d)  # 1 2 3 4
```

- Nested tuples are used to represent matrices, tables, and grids
- Each inner tuple can be accessed using its index
- Double indexing is used to access individual elements
- Tuples inside tuples are also immutable

Suggested Programs

1. Finding the Minimum, Maximum, and Mean of Values Stored in a Tuple

Purpose: Find min, max, and mean of numbers stored in a tuple

```
numbers = (25, 10, 45, 30, 15)

# Using built-in functions
minimum = min(numbers)
maximum = max(numbers)
mean = sum(numbers) / len(numbers)

print("Minimum:", minimum)   # 10
print("Maximum:", maximum)   # 45
print("Mean:", mean)         # 25.0
```

2. Finding Minimum, Maximum, and Mean Without Built-in Functions

```
numbers = (25, 10, 45, 30, 15)

minimum = numbers[0]
maximum = numbers[0]
total = 0

for num in numbers:
    if num < minimum:
        minimum = num
    if num > maximum:
        maximum = num
    total += num

mean = total / len(numbers)

print("Minimum:", minimum)   # 10
```

```
print("Maximum:", maximum)   # 45
print("Mean:", mean)         # 25.0
```

3. Linear Search on a Tuple of Numbers

Purpose: Search for an element in a tuple by checking each element one by one

```
numbers = (10, 25, 30, 45, 50)
search = 30
found = False
position = -1

for i in range(len(numbers)):
    if numbers[i] == search:
        found = True
        position = i
        break

if found:
    print(search, "found at index", position)
else:
    print(search, "not found")
# Output: 30 found at index 2
```

4. Counting the Frequency of Elements in a Tuple

Purpose: Count how many times each element appears in a tuple

```
numbers = (10, 20, 10, 30, 20, 10, 40)

# Method 1: Using count()
visited = []
for num in numbers:
    if num not in visited:
        print(num, "occurs", numbers.count(num), "times")
        visited.append(num)

# Output:
# 10 occurs 3 times
# 20 occurs 2 times
# 30 occurs 1 times
# 40 occurs 1 times
```

5. Counting Frequency Using Dictionary

```
numbers = (10, 20, 10, 30, 20, 10, 40)

frequency = {}
for num in numbers:
    if num in frequency:
        frequency[num] += 1
    else:
        frequency[num] = 1

for key, value in frequency.items():
    print(key, "occurs", value, "times")
```

6. Counting Frequency Using Nested Loops

```
numbers = (10, 20, 10, 30, 20, 10, 40)
n = len(numbers)
counted = [False] * n

for i in range(n):
    if counted[i]:
        continue
    count = 1
    for j in range(i + 1, n):
        if numbers[i] == numbers[j]:
            count += 1
            counted[j] = True
    print(numbers[i], "occurs", count, "times")
```

Quick Reference Table

Method / Function	Purpose	Returns
`len()`	Number of elements	Integer
`tuple()`	Create tuple from iterable	Tuple
`count()`	Count occurrences	Integer

Method / Function	Purpose	Returns
`index()`	Find position	Integer
`sorted()`	Return sorted list	List
`min()`	Smallest element	Element
`max()`	Largest element	Element
`sum()`	Sum of elements	Number

Important Board Exam Tips Must Know

1. **Tuple Immutability:** Tuples cannot be changed in place. No methods like append(), insert(), remove(), or pop() exist for tuples.
2. **Single Element Tuple:** Always use a trailing comma: `(5,)` is a tuple, but `(5)` is just an integer.
3. **Indexing:** Remember that indexing starts from 0 in Python.
4. **Negative Indexing:** -1 is the last element, -2 is second last, etc.
5. **Slicing Rules:**
   - `tuple[start:end]` — includes start, excludes end
   - `tuple[:end]` — from beginning to end-1
   - `tuple[start:]` — from start to end
   - `tuple[::-1]` — reverses the tuple
6. **Membership Operators:** `in` and `not in` are case-sensitive for strings.
7. **Tuple Assignment:**
   - Number of variables must match number of elements
   - Use `*` to collect multiple values into a list
   - Useful for swapping variables: `a, b = b, a`
8. **sorted() vs tuple.sort():**
   - Tuples have no `sort()` method (immutable)
   - `sorted()` returns a list, not a tuple
   - Use `tuple(sorted(tup))` to get a sorted tuple
9. **Nested Tuples:** Use double indexing like `nested[i][j]` to access elements.
10. **Common Errors to Avoid:**

- Forgetting the comma in single element tuple
- Trying to modify a tuple (raises TypeError)
- Using `index()` on an element not in the tuple (raises ValueError)
- Confusing tuple assignment with list assignment
- Using `sorted()` and expecting a tuple in return
